AI Drug Toxicity Analyzer
with Explainable Molecular Insights

Hackathon Project Document

Track A — Drug Toxicity Prediction

Table of Contents

1. Problem Statement
2. Solution Overview
3. Technology Stack
4. Installation & Setup Guide
5. Project Folder Structure
6. Code Writing Style & Conventions
7. Feature Breakdown
8. Application Flow
9. Model Training Pipeline
10. Deployment
11. Judging Criteria Alignment

1. Problem Statement

In modern pharmaceutical research, determining the toxicity of chemical compounds is a critical but time-consuming process. Traditional lab-based toxicity testing is expensive, slow, and raises ethical concerns. The challenge in Track A (Drug Toxicity) of this hackathon is:

> **Given a chemical compound's molecular structure (expressed as a SMILES string), predict whether it is toxic or non-toxic — and explain WHY.**

This problem is important because:

- Thousands of compounds fail in clinical trials due to unexpected toxicity
- AI-powered early prediction can save millions in drug development costs
- Explainable predictions help chemists and researchers act on results
- Regulatory bodies increasingly require interpretable AI models in healthcare

2. Solution Overview

We build a full-stack AI-powered web application: the AI Drug Toxicity Analyzer. It takes a SMILES molecular string as input and outputs a toxicity prediction, confidence score, risk rating, and a human-readable explanation of why the compound is predicted to be toxic or safe.

2.1 Key Capabilities

- SMILES input with real-time molecular descriptor computation using RDKit
- Toxicity prediction (Toxic / Non-Toxic) using an XGBoost model
- Probability score and Risk Score (0-100) for severity grading
- Explainable AI panel using SHAP values showing top contributing features
- Auto-generated toxicity explanation report in plain English
- Visual comparison dashboard of toxic vs non-toxic compound distributions
- Substructure-based toxicity alerts (halogen groups, aromatic rings, etc.)
- Similar molecule finder from dataset for research context

2.2 Architecture Summary

- Frontend: React.js (interactive UI, charts, SMILES input)
- Backend: Python + FastAPI (ML model serving, RDKit descriptor computation)
- ML Model: XGBoost trained on Tox21 / ZINC dataset
- Explainability: SHAP library
- Database: MongoDB (store past predictions, compound records)
- Visualization: Recharts / Plotly

3. Technology Stack

Layer	Technology	Purpose
Frontend	React.js + Tailwind CSS	UI, interactive inputs, charts
Backend API	Python + FastAPI	ML inference, RDKit, SHAP
ML Model	XGBoost + Scikit-learn	Toxicity classification
Explainability	SHAP	Feature importance explanations
Chemistry	RDKit	Molecular descriptor computation
Database	MongoDB + Mongoose	Storing predictions & compounds
Data Processing	Pandas + NumPy	Dataset manipulation
Visualization	Recharts / Plotly	Charts in frontend
HTTP Layer	Axios + Express.js	REST API calls
Environment	Python venv + Node.js	Dependency isolation

4. Installation & Setup Guide

Follow these steps in order. You will need Python 3.10+, Node.js 18+, and MongoDB installed.

4.1 System Prerequisites

Tool	Version	Download Link
Python	3.10 or higher	https://python.org/downloads
Node.js	18 or higher	https://nodejs.org
MongoDB	6.0 or higher	https://mongodb.com/try/download/community
Git	Any recent	https://git-scm.com

4.2 Python Backend Setup

Open your terminal and run the following commands:

```
# Step 1: Create and activate a virtual environment
python -m venv venv
source venv/bin/activate        # On Windows: venv\Scripts\activate

# Step 2: Install Python dependencies
pip install fastapi uvicorn xgboost scikit-learn shap pandas numpy rdkit-pypi
pymongo python-dotenv

# Step 3: Verify RDKit installation
python -c "from rdkit import Chem; print('RDKit OK')"
```

4.3 React Frontend Setup

```
# Navigate to frontend directory
cd frontend
npm install
npm install axios recharts tailwindcss @tailwindcss/forms
```

4.4 MongoDB Setup

Start MongoDB locally or use MongoDB Atlas (free cloud cluster):

- Local: Run 'mongod' in terminal after installation
- Cloud: Create free cluster at mongodb.com/atlas and copy your connection string

```
# Add to your .env file:
MONGO_URI=mongodb://localhost:27017/drugtoxicity
```

4.5 Environment Variables

Create a .env file in the backend folder:

```
MONGO_URI=mongodb://localhost:27017/drugtoxicity
```

```
MODEL_PATH=./models/xgboost_toxicity.pkl
PORT=8000
```

5. Project Folder Structure

The project is divided into three main parts: frontend (React), backend (Python/FastAPI), and ml_pipeline (model training scripts).

```
drug-toxicity-analyzer/
│
├── frontend/                          # React.js Application
│   ├── public/
│   │   └── index.html
│   ├── src/
│   │   ├── components/
│   │   │   ├── SMILESInput.jsx         # SMILES string input box
│   │   │   ├── PredictionResult.jsx    # Shows toxic/non-toxic result
│   │   │   ├── SHAPExplainer.jsx       # SHAP feature bar chart
│   │   │   ├── RiskScore.jsx           # Circular risk gauge
│   │   │   ├── MoleculeViewer.jsx      # 2D molecule structure
│   │   │   ├── ComparisonDashboard.jsx # Toxic vs non-toxic charts
│   │   │   └── SimilarMolecules.jsx    # Similar compound finder
│   │   ├── pages/
│   │   │   ├── Home.jsx                # Landing page
│   │   │   ├── Analyzer.jsx            # Main analysis page
│   │   │   └── History.jsx             # Past prediction history
│   │   ├── services/
│   │   │   └── api.js                  # Axios API calls
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── tailwind.config.js
│
├── backend/                           # Python FastAPI Server
│   ├── main.py                        # FastAPI app entry point
│   ├── routes/
│   │   ├── predict.py                 # POST /predict endpoint
│   │   ├── history.py                 # GET /history endpoint
│   │   └── molecules.py               # GET /similar endpoint
│   ├── services/
│   │   ├── descriptor_engine.py       # RDKit descriptor computation
│   │   ├── model_service.py           # Load XGBoost, run inference
│   │   ├── shap_service.py            # SHAP explanation generator
│   │   ├── report_generator.py        # Auto-generates text report
│   │   └── substructure_checker.py    # Toxic group detector
│   ├── models/
│   │   └── xgboost_toxicity.pkl       # Trained model file
│   ├── db/
│   │   └── mongo_client.py            # MongoDB connection
│   ├── schemas/
│   │   └── prediction_schema.py       # Pydantic request/response models
│   ├── .env
│   └── requirements.txt
│
├── ml_pipeline/                       # Model Training Scripts
│   ├── data/
│   │   └── tox21_dataset.csv          # Raw dataset
│   ├── notebooks/
│   │   └── EDA.ipynb                  # Exploratory Data Analysis
│   ├── preprocess.py                  # Data cleaning + descriptor generation
│   ├── train.py                       # Model training + evaluation
│   ├── evaluate.py                    # Metrics, confusion matrix
│   └── feature_importance.py          # SHAP global analysis
```

```
│
├── README.md
└── .gitignore
```

6. Code Writing Style & Conventions

6.1 Python (Backend & ML)

- Use snake_case for all variable and function names
- Use type hints on all function signatures
- One class or one responsibility per file — keep files focused
- All environment variables loaded from .env using python-dotenv
- Use Pydantic models for API request/response validation
- Return consistent JSON structure: { success, data, message }

Example — descriptor_engine.py:

```
from rdkit import Chem
from rdkit.Chem import Descriptors

def compute_descriptors(smiles: str) -> dict:
    mol = Chem.MolFromSmiles(smiles)
    if mol is None:
        raise ValueError(f"Invalid SMILES: {smiles}")
    return {
        "mol_weight": Descriptors.MolWt(mol),
        "logP": Descriptors.MolLogP(mol),
        "hbd": Descriptors.NumHDonors(mol),
        "hba": Descriptors.NumHAcceptors(mol),
        "tpsa": Descriptors.TPSA(mol),
    }
```

6.2 JavaScript / React (Frontend)

- Use functional components with React Hooks only
- Use camelCase for variables and functions, PascalCase for components
- Keep all API calls in src/services/api.js — never call axios from components directly
- Use Tailwind CSS utility classes for all styling — no custom CSS files
- One component = one file, named same as component

Example — api.js:

```
import axios from 'axios';
const BASE_URL = 'http://localhost:8000';

export const predictToxicity = async (smiles) => {
  const res = await axios.post(`${BASE_URL}/predict`, { smiles });
  return res.data;
};
```

6.3 MongoDB Schema (via Mongoose / pymongo)

- Collection: predictions
- Fields: _id, smiles, descriptors, prediction, probability, risk_score, shap_values, report, created_at
- Always index on created_at for history queries

7. Feature Breakdown

7.1 Toxicity Prediction Model

The core ML model is an XGBoost classifier trained on the Tox21 dataset. Input features are RDKit molecular descriptors.

Feature	Description	Impact
Molecular Weight	Mass of the compound	Medium
LogP	Lipophilicity (fat solubility)	High
HBD	Hydrogen bond donors	High
HBA	Hydrogen bond acceptors	Medium
TPSA	Topological polar surface area	High
QED	Drug-likeness score	High
Aromatic Rings	Count of aromatic ring systems	Medium

7.2 SHAP Explainability

After every prediction, SHAP (SHapley Additive exPlanations) values are computed to explain which molecular features pushed the model toward toxic or non-toxic.

- Positive SHAP value = feature increases toxicity probability
- Negative SHAP value = feature decreases toxicity probability
- Top 5 features are shown in a horizontal bar chart in the UI

7.3 Auto-Generated Toxicity Report

The report_generator.py service takes SHAP values and descriptor values and generates a natural language explanation. Example output:

> *"This compound is predicted TOXIC (89% confidence). Primary contributors: high lipophilicity (logP = 5.2), presence of halogen substructure, and low polarity (TPSA = 18.3). High LogP values are associated with bioaccumulation in fatty tissues. Risk Score: 82/100 (High Risk)."*

7.4 Risk Score System

Instead of a binary toxic/non-toxic label, a Risk Score from 0-100 is computed:

- 0-30: Low Risk (Safe)
- 31-60: Moderate Risk (Caution)
- 61-80: High Risk (Likely Toxic)
- 81-100: Critical Risk (Highly Toxic)

8. Application Flow

The complete flow from user input to final output:

1. User opens the React app and navigates to the Analyzer page
2. User pastes a SMILES string (e.g., CC(=O)Oc1ccccc1C(=O)O for Aspirin)
3. React calls POST /predict via Axios with the SMILES string
4. FastAPI backend receives request and validates SMILES using RDKit
5. RDKit computes molecular descriptors (LogP, MW, HBD, HBA, TPSA, etc.)
6. Descriptors are passed to the XGBoost model for toxicity prediction
7. SHAP values are computed to explain the prediction
8. Substructure checker scans for known toxic functional groups
9. Report generator assembles a natural language explanation
10. Risk score (0-100) is calculated from the probability output
11. Full response (prediction + score + SHAP + report) is returned to React
12. React renders: result badge, risk gauge, SHAP chart, report text, similar molecules
13. Prediction is saved to MongoDB for history tracking

9. Model Training Pipeline

9.1 Dataset

- Primary: Tox21 Dataset (8,014 compounds, 12 toxicity targets)
- Supplementary: ZINC database (for QED and SAS features)
- Focus on NR-AR target (androgen receptor — most predictable)

9.2 Training Steps

Run these scripts in order from the ml_pipeline/ directory:

```
# Step 1: Generate descriptors from SMILES in dataset
python preprocess.py

# Step 2: Train XGBoost model
python train.py

# Step 3: Evaluate model performance
python evaluate.py

# Step 4: Generate global SHAP analysis plots
python feature_importance.py
```

9.3 Expected Model Metrics

Metric	Target Value	Notes
Accuracy	> 80%	On held-out test set
ROC-AUC	> 0.85	Key metric for imbalanced data
Precision	> 0.78	Toxic class
Recall	> 0.75	Toxic class (catching true positives)

9.4 train.py Skeleton

```
import pandas as pd
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
import pickle

df = pd.read_csv('data/descriptors.csv')
X = df.drop(columns=['label', 'smiles'])
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = xgb.XGBClassifier(
```

```
    n_estimators=300,
    max_depth=6,
    learning_rate=0.05,
    scale_pos_weight=3,   # handle class imbalance
    use_label_encoder=False,
    eval_metric='auc'
)
model.fit(X_train, y_train)

auc = roc_auc_score(y_test, model.predict_proba(X_test)[:,1])
print(f'Test AUC: {auc:.4f}')

with open('models/xgboost_toxicity.pkl', 'wb') as f:
    pickle.dump(model, f)
```

10. Deployment

10.1 Running Locally

```
# Terminal 1 — Start Python backend
cd backend
source ../venv/bin/activate
uvicorn main:app --reload --port 8000

# Terminal 2 — Start React frontend
cd frontend
npm run dev

# Terminal 3 — Start MongoDB (if local)
mongod
```

10.2 Access Points

- React App: http://localhost:5173
- FastAPI Docs (Swagger UI): http://localhost:8000/docs
- MongoDB: mongodb://localhost:27017/drugtoxicity

11. Judging Criteria Alignment

Based on the judging priorities for this hackathon, here is how each project feature maps to scoring areas:

Judging Criterion	How We Address It	Features
Interpretation	SHAP explainability + report	SHAP panel, text report
Real-world Usability	Interactive SMILES input UI	Analyzer page, live API
Biological Meaning	Substructure alerts, descriptor chemistry	Halogen/aromatic detection
Clean Presentation	React UI + Tailwind + charts	Recharts, risk gauge
Innovation	ZINC feature engineering, risk score	QED, SAS, 0-100 score
Technical Depth	XGBoost + RDKit + SHAP pipeline	Full ML pipeline

By combining technical rigor (XGBoost + RDKit + SHAP) with an excellent user experience (React + clear visuals + plain English reports), this project is positioned to stand out from teams that only build a basic model without explanation or UI.

Good luck! Build it, explain it, and win it.